

Service Engineering & Identification

Lost and Found Management System

Deliverable #5 — Part 1

Spring 2026

Course: Service-Oriented Software Engineering (C-SW311)

Deliverable: #5 — Part 1: Service Engineering & Identification

Project: Lost and Found Management System

Table of Contents

- **Section 1:** Understanding the Existing Project
- **Section 2:** Service Candidate Identification
- **Section 3:** Service Classification
 - 3.1 Utility Services
 - 3.2 Business Services
 - 3.3 Coordination Services
- **Section 4:** Final Deliverable Tables
- **Section 5:** Final Recommendation

Part 1: Service Engineering & Identification

Lost and Found Management System

Course: Service-Oriented Software Engineering (C-SW311)

Deliverable #5 — Part 1

Section 1: Understanding the Existing Project

1.1 System Overview

The Lost and Found Management System is a **Java Swing desktop application** backed by a **Microsoft SQL Server** database. It manages the full lifecycle of lost and found items within a university or organization, from item logging to student collection. The system enforces **role-based access control** across three actor types: **Admin**, **Officer**, and **Student**.

Technology Stack (evidence from repository):

- Language: Java 11+ (JDK required, build.xml uses Apache Ant)
- UI Framework: Java Swing (src/Boundary/ LoginWindow.java, AdminDashboard.java, etc.)
- Database: Microsoft SQL Server (mssql-jdbc-13.2.1.jre11.jar, LostAndFoundDB.bak)
- Architecture: MVC — Boundary / Controller / DAO layers (src/Boundary, src/controller, src/DAO)
- Design Patterns: Factory, Singleton, Observer, Bridge, Strategy, Decorator, Template Method, Flyweight, Adapter

1.2 Database Tables (Inferred from DAO SQL Queries)

Table	Key Columns	Purpose
[User]	UserID, Name, Email, PasswordHash, Role	Base user record
ADMIN	UserID, Permissions	Admin role sub-table
OFFICER	UserID, SecurityBadgeNumber	Officer role sub-table
STUDENT	UserID, AcademicYear	Student role sub-table
FOUND_ITEM	ItemID, Title, Description, Category, Color, Location, Photo (VARBINARY), Status, DateUploaded, UploadedByUserID	Found items inventory
CLAIM	ClaimID, Status, DateInitiated, ClaimedByUserID, ItemID	Claim requests
HANDOVER_LOG	LogID, ClaimID, Timestamp	Physical handover audit trail

Item Status Lifecycle (from ItemController.java CHECK constraint comment):
Found —> Processing Claim —> Collected / Archived

1.3 Core Entities

5/9/26, 8:50 PM

Part 1: Service Engineering & Identification

Entity Class	Key Fields	Role
Item	itemID, title, description, category, status, location, photoPath, dateFound, officerID, officerName, itemType (Flyweight)	Represents a found item
Claim	claimID, status, requestDate, studentID, itemID	Represents a student's claim
User	userID, roleID, email	Base user
HandoverLog	logID, claimID, timestamp	Physical handover record
NotificationRecord	userId, message, channel, timestamp	Notification audit
ItemTypeFlyweight	category, status	Shared item metadata (Flyweight)

1.4 Controllers and Responsibilities

Controller	Key Methods	Responsibility
ItemController	registerNewItem(), deleteItem(), modifyItem(), validateItem(), getItemDetails()	Found item CRUD; triggers Observer + Bridge notifications on upload
ClaimController	startClaimProcess(), processClaimWithStrategy(), updateClaimStatus(), processResponse(), validateDecision()	Claim lifecycle; uses Strategy Pattern (Strict / Fast / Manual)
MatchingController	performSmartSearch(), performStrategySearch(), applyMatchingLogic()	Item search; uses Decorator Pattern (multi-filter) + Strategy Pattern
ReportController	createWeeklyReport(), fetchLostItems(), fetchClaimedItems(), buildWeeklyReportBridge()	Report generation; uses Bridge + Template Method patterns
HandoverController	logCollection(), verifyCollectionCredentials()	Physical item handover; uses Template Method
ArchiveController	archiveExpiredItems()	Batch archiving of old items
UserController	changeUserRole(), authorizeChange()	User role management

1.5 Boundary (UI) Screens

Screen Class	Role	Features
LoginWindow	All	Username/password auth, role-based routing, theme toggle
AdminDashboard	Admin	Central hub for admin operations
OfficerDashboard	Officer	Found item upload, claim management hub
UserAdminPanel	Admin	Add, delete, update user roles
UploadForm	Officer	Register new found item with photo, category, location
SearchWindow	Student/Officer	Advanced search with filters (category, status, location, date)
ClaimSubmissionWindow	Student	Submit a claim for an item
OfficerClaimsWindow	Officer	Review and approve/reject pending claims
ClaimResponseWindow	Officer	Respond to individual claim
HandoverWindow	Officer	Log physical item handover/collection
ReportWindow	Admin/Officer	View weekly/claims/found-item reports
NotificationWindow	All	View in-app notifications
DeleteItemWindow	Officer/Admin	Delete item records
EditItemWindow	Officer	Edit item details
ArchiveDashboard	Admin	Archive expired items
ItemDetailsWindow	All	View item details

1.6 Notification Infrastructure

The system implements a **Bridge Pattern** for multi-channel notification delivery (src/bridge/notification/):

- NotificationSender (interface): InAppSender, EmailSender, SmsSender
- Notification (abstraction): ItemStatusNotification, ClaimNotification, GeneralNotification
- Additionally, an **Observer Pattern** (behaviouralpatterns/observer/case1_itemposting/NotificationManager) broadcasts item posting events to subscribed users.

Authentication: Plain-text password comparison currently in DBConnection.authenticateUser(). SHA-256 hashPassword() method exists but is commented out â€” a critical security gap.

Section 2: Service Candidate Identification

Using Ian Sommerville's Chapter 18 criteria, each candidate service was evaluated against:

1. Does it represent a single logical entity or process?
2. Is it used by multiple roles?
3. Is it independent with well-defined inputs/outputs?
4. Does it require its own state or database?
5. Could external clients use it?
6. Does it have distinct non-functional requirements (security, performance)?

2.1 Evidence-Based Identification from the Repository

The following business functions and technical modules are strong service candidates:

#	System Component	Source Evidence	Reason for Candidacy
1	Authentication	DBConnection.authenticateUser(), SessionManager, LoginWindow	Used by all 3 roles; single responsibility; stateless input/output
2	User Management	DBConnection.addUser/deleteUser/updateUserRole(), UserAdminPanel, UserController	Distinct admin operation; independent; reusable
3	Found Item Registration	ItemController.registerNewItem(), UploadForm, ItemDataAccess.saveItem()	Core officer function; triggers multiple downstream events
4	Item Search & Filter	MatchingController, SearchWindow, Decorator + Strategy patterns	Used by both Students and Officers; decoupled; high reusability
5	Claim Management	ClaimController, ClaimDataAccess, OfficerClaimsWindow, Strategy Pattern	Complex lifecycle with 3 strategies; used by 2 roles
6	Notification Delivery	bridge/notification/ (InApp/Email/SMS senders), NotificationManager (Observer)	Cross-cutting; used by Item, Claim, and Handover modules
7	Reporting & Statistics	ReportController, ReportWindow, DBConnection.getReportStats(), Bridge + Template Method	Admin/Officer use; distinct output formats; analytical
8	Handover / Collection	HandoverController, HandoverDataAccess, HandoverWindow, Template Method	Final workflow step; audit trail; officer-only
9	Archive Management	ArchiveController, ArchiveDashboard	Scheduled batch process; admin-only; independent lifecycle
10	Item Matching / Smart Search	MatchingController.performSmartSearch(), Decorator chain	Algorithmic; can serve mobile/web clients externally
11	Category & Status Management	ItemTypeFlyweight, ItemTypeFactory, category/status CHECK in DB	Shared metadata; referenced by all item operations

Section 3: Service Classification

3.1 Utility Services

Utility services provide low-level, reusable technical capabilities that support all business operations.

US-1: Authentication Service

A. Description:
Verifies user credentials (username + password) against the [User] table in SQL Server and returns the authenticated user's role (Admin / Officer / Student). Currently implemented in DBConnection.authenticateUser() and SessionManager.

- B. Why a Service Candidate (Chapter 18 reasoning):**
- Used by ALL three roles â€” every system interaction requires prior authentication
 - Completely stateless (input: credentials â†’ output: role + userID)
 - Naturally independent from all business logic
 - External clients (mobile app, web portal) would need the same authentication gateway

- C. Functional Requirements:**
- POST /auth/login â€” validate credentials, return JWT token + role
 - POST /auth/logout â€” invalidate session token
 - GET /auth/validate â€” verify token validity

D. Non-Functional Requirements:

Attribute	Requirement
Security	HTTPS only; passwords hashed with SHA-256 or bcrypt (SHA-256 implementation exists but is disabled in the repository)
Performance	Response < 200ms for login
Availability	99.9% uptime â€” system is unusable without it
Reliability	Zero false positives; failed logins must be logged
Scalability	Stateless design allows horizontal scaling
Maintainability	Token-based auth (JWT) decouples session from server state

E. Suggested REST Resources:

POST	/api/auth/login	â€” Authenticate user, return token
POST	/api/auth/logout	â€” Invalidate session
GET	/api/auth/validate	â€” Validate token
GET	/api/auth/me	â€” Return current user profile from token

US-2: Notification Service

A. Description:
Delivers notifications to users across multiple channels (In-App, Email, SMS). Currently implemented via the Bridge Pattern in src/bridge/notification/ with senders InAppSender, EmailSender, SmsSender, and notification types ItemStatusNotification, ClaimNotification, GeneralNotification. The Observer pattern (NotificationManager) triggers broadcasts on item posting.

B. Why a Service Candidate:

- Cross-cutting concern: triggered by `ItemController` (item upload), `ItemDataAccess` (item deletion), and implicitly by claim status changes
- Completely independent of business data â€” only needs a `userID` + message + channel
- Can serve all current and future system modules
- External systems (email relay, SMS gateway) are already abstracted via the Bridge interface

C. Functional Requirements:

- Send notification to a specific user by channel
- Broadcast to all subscribers when a new item is posted
- Store notification history (`NotificationRecord` entity)
- Retrieve notification history for a user

D. Non-Functional Requirements:

Attribute	Requirement
Security	Only authenticated services can trigger notifications
Performance	Async delivery â€” must not block main transaction
Availability	Degraded gracefully â€” main operations proceed if notification fails (already handled in <code>ItemController</code> with try-catch)
Reliability	Retry mechanism for email/SMS failures
Scalability	Queue-based delivery for broadcasts
Maintainability	Bridge pattern already isolates channel implementations

E. Suggested REST Resources:

POST	/api/notifications/send	â€” Send notification to a user
POST	/api/notifications/broadcast	â€” Broadcast to all subscribers
GET	/api/notifications/{userID}	â€” Get notification history for user
PUT	/api/notifications/{id}/read	â€” Mark as read

US-3: Password & Security Service**A. Description:**

Provides password hashing, validation, and security utilities. The `hashPassword()` method using SHA-256 already exists in `DBConnection.java` but is currently commented out â€” the system stores plaintext passwords. This service formalises and enforces secure credential storage.

B. Why a Service Candidate:

- Security is a cross-cutting concern used by both Authentication and User Management
- Should be independently upgradeable (e.g., SHA-256 â†’ bcrypt) without touching business logic
- Distinct non-functional requirements (FIPS compliance, algorithm agility)

C. Functional Requirements:

- Hash a plaintext password
- Verify a plaintext password against a stored hash
- Generate secure tokens

D. Non-Functional Requirements:

Attribute	Requirement
Security	Must use bcrypt or SHA-256 minimum; never log plaintext passwords
Performance	< 100ms for hash operations
Maintainability	Algorithm must be configurable without code changes

E. Suggested REST Resources:

POST	/api/security/hash	â€” Hash a password (internal only)
POST	/api/security/verify	â€” Verify password against hash (internal only)

3.2 Business Services

Business services directly implement the domain logic of the Lost and Found system.

BS-1: Found Item Service**A. Description:**

Manages the complete CRUD lifecycle of found items in the `FOUND_ITEM` table. Evidence: `ItemController.registerNewItem()`, `ItemDataAccess.saveItem/updateItemRecord/removeItem/getAllItems/findItemByID()`, `UploadForm.java`, `EditItemWindow.java`, `DeleteItemWindow.java`.

B. Why a Service Candidate:

- Core domain entity â€” every other service depends on it
- Used by Officers (upload, edit, delete) and by Students (view)
- Database-backed with its own table and lifecycle states
- Can be consumed by external clients (mobile app for officers in the field)

C. Functional Requirements:

- Register a new found item (with photo, category, location, date)
- Update item details (title, description, category, location, photo, status)
- Delete an item (cascades to associated claims)

- Retrieve item details by ID
- Retrieve all items with optional status filter
- Update item status (Found → Processing Claim → Collected / Archived)

D. Non-Functional Requirements:

Attribute	Requirement
Security	Only Officers/Admins can create, update, or delete items; Students read-only
Performance	Item listing < 500ms; photo upload < 2 seconds
Reliability	Database transactions for delete (cascading claims)
Scalability	Photo storage should be externalized (file system or blob storage)
Maintainability	Status transitions enforced by service, not by UI

E. Suggested REST Resources:

GET	/api/items	â€” List all items (filterable by status)
GET	/api/items/{id}	â€” Get item details
POST	/api/items	â€” Register new found item
PUT	/api/items/{id}	â€” Update item details
DELETE	/api/items/{id}	â€” Delete item
PATCH	/api/items/{id}/status	â€” Update item status
POST	/api/items/{id}/photo	â€” Upload item photo

BS-2: Search & Matching Service

A. Description:

Provides multi-criteria search and intelligent matching of found items. Evidence: `MatchingController` with Decorator Pattern (category + status + location + date filters) and Strategy Pattern (search by name / category / location). `SearchWindow.java` demonstrates the rich UI built around this capability.

B. Why a Service Candidate:

- Used by both Students (to find their lost item) and Officers (to check existing inventory)
- Complex algorithmic logic independent from data storage
- `Item.matchScore()` stub indicates intent for scoring-based matching
- A mobile app or web portal would need the same search capability

C. Functional Requirements:

- Full-text search by keyword (title, description)
- Filter by category, status, location, date range
- Strategy-based focused search (by name / category / location)
- Return scored/ranked results
- Return items with status Found or Processing Claim only for student search

D. Non-Functional Requirements:

Attribute	Requirement
Security	Read-only; accessible to authenticated Students and Officers
Performance	Results returned < 300ms for typical queries
Scalability	Indexing on Title, Description, Category, Location columns
Reliability	Empty result set returned (never error) when no match
Maintainability	Decorator chain allows adding new filters without changing core logic

E. Suggested REST Resources:

GET	/api/search?q={keyword}&category={cat}&location={loc}&status={s}&from={date}&to={date}	
GET	/api/search/strategy?type=name category location&keyword={kw}	
GET	/api/search/match/{itemID}	â€” Find potential matches for a specific item

BS-3: Claim Management Service

A. Description:

Manages the full lifecycle of a claim: submission by student, review by officer, approval/rejection decision, and status tracking. Evidence: `ClaimController` (with Strategy Pattern for Strict / Fast / Manual processing), `ClaimDataAccess` (insert, getAllClaims, getPendingClaims, updateClaimStatus, getClaimById), `ClaimSubmissionWindow`, `OfficerClaimsWindow`, `ClaimResponseWindow`.

B. Why a Service Candidate:

- Multi-role service: Students submit, Officers process
- Complex workflow with multiple processing strategies
- Maintains its own state in the CLAIM table
- The Strategy Pattern (`StrictClaimStrategy`, `FastClaimStrategy`, `ManualClaimStrategy`) proves interchangeable business rules
- Triggers downstream Handover and Notification services

C. Functional Requirements:

- Student submits a claim for an item
- Retrieve all pending claims (for Officer dashboard)
- Retrieve claims by student (for Student history)
- Approve/reject a claim with a selected strategy (Strict, Fast, Manual)
- Update item status to Processing Claim upon claim submission
- Retrieve claim details by ID

D. Non-Functional Requirements:

Attribute	Requirement
Security	Only the student who owns a claim can view it; only Officers can approve/reject
Performance	Claim submission < 400ms
Reliability	Atomic "claim insert and item status update in single transaction
Scalability	Queue pending claims for asynchronous officer review
Maintainability	New claim strategies added without modifying ClaimController

E. Suggested REST Resources:

POST	/api/claims	" Submit a new claim (Student)
GET	/api/claims	" List all claims (Officer/Admin)
GET	/api/claims/pending	" List pending claims (Officer)
GET	/api/claims/{id}	" Get claim details
PUT	/api/claims/{id}/approve	" Approve claim with strategy
PUT	/api/claims/{id}/reject	" Reject claim
GET	/api/claims/student/{studentID}	" Get claims by student

BS-4: User Management Service**A. Description:**

Manages user accounts including creation, deletion, and role assignment. Evidence: DBConnection.addUser(), DBConnection.deleteUser(), DBConnection.updateUserRole(), DBConnection.getAllUsersWithRoles(), DBConnection.searchUserById(), UserAdminPanel.java, UserController.java, UserDataAccess.java.

B. Why a Service Candidate:

- Exclusively admin-facing operation with clear boundary
- Independent from item/claim business logic
- Manages the multi-table role hierarchy ([User], ADMIN, OFFICER, STUDENT)

C. Functional Requirements:

- Create a new user with a specified role
- Delete a user (cascades claims and items)
- Update a user's role (transactional across sub-tables)
- List all users with roles
- Search user by ID

D. Non-Functional Requirements:

Attribute	Requirement
Security	Admin-only; all operations require Admin token
Reliability	Role update is transactional " remove from old role table, insert into new
Maintainability	Role hierarchy isolated in sub-tables; extendable to new roles

E. Suggested REST Resources:

GET	/api/users	" List all users with roles
GET	/api/users/{id}	" Get user by ID
POST	/api/users	" Create new user
PUT	/api/users/{id}/role	" Update user role
DELETE	/api/users/{id}	" Delete user

BS-5: Reporting & Statistics Service**A. Description:**

Generates operational reports for management. Evidence: ReportController (weekly report, lost items report, claimed items report via Bridge Pattern and Template Method Pattern), ReportWindow.java, DBConnection.getReportStats() returning counts for Found / Claims / Collected / Not Collected.

B. Why a Service Candidate:

- Distinct analytical function used by Admin and Officers
- The Bridge Pattern already separates report data from format (could output JSON, CSV, PDF)
- report_adapter.csv in the repository confirms external data export is already in use
- Template Method isolates report structure from specific data queries

C. Functional Requirements:

- Get system statistics (counts: found, claimed, collected, not-collected)
- Generate weekly item report (last 7 days)
- Generate lost items report (items with status Found or Not Collected)
- Generate claimed items report (all claims)
- Export reports in multiple formats (JSON / CSV)

D. Non-Functional Requirements:

Attribute	Requirement
Security	Admin and Officer access only
Performance	Report generation < 1 second for typical dataset
Reliability	Must handle empty datasets gracefully

Attribute	Requirement
Scalability	For large datasets, paginate or use async report generation
Maintainability	Bridge Pattern allows adding PDF/HTML output without changing data logic

E. Suggested REST Resources:

GET	/api/reports/stats	â€” Get system statistics summary
GET	/api/reports/weekly	â€” Weekly item report
GET	/api/reports/lost-items	â€” Lost/found items report
GET	/api/reports/claims	â€” Claims report
GET	/api/reports/export?format=csv	â€” Export report as CSV

BS-6: Archive Service**A. Description:**

Batch-archives items that have exceeded a configurable time threshold without being collected. Evidence: ArchiveController (threshold date, batch size, archiveExpiredItems() stub), ArchiveDashboard.java.

B. Why a Service Candidate:

- Lifecycle management distinct from normal CRUD
- Can be triggered on a schedule (cron job) independently
- Admin-only; no student or officer interaction needed

C. Functional Requirements:

- Archive all items older than a configurable threshold date
- Set archive threshold and batch size
- Return count of archived items

D. Non-Functional Requirements:

Attribute	Requirement
Security	Admin-only
Performance	Batch processing; must not block the main application
Reliability	Atomic batch â€” all or none
Scalability	Batch size configurable to avoid long-running transactions

E. Suggested REST Resources:

POST	/api/archive/run	â€” Trigger archive process
GET	/api/archive/config	â€” Get archive configuration
PUT	/api/archive/config	â€” Update threshold and batch size
GET	/api/archive/history	â€” Get archive run history

3.3 Coordination Services

Coordination services orchestrate multiple business services to fulfill end-to-end workflows.

CS-1: Item Claim Workflow Service**A. Description:**

Orchestrates the complete end-to-end claim workflow: Student searches for item â†’ submits claim â†’ item status updated to Processing Claim â†’ Officer reviews â†’ approves or rejects â†’ notification sent â†’ if approved, Handover Service triggered. This spans MatchingController, ClaimController, ItemController, HandoverController, and NotificationService.

B. Why a Service Candidate:

- No single existing controller owns the full workflow
- Requires coordinating state changes across FOUND_ITEM and CLAIM tables atomically
- Multiple external events (notifications, handover logging) must be sequenced correctly
- The Activity Diagram (ClaimProcess_ActivityDiagram.mdj) and activity diagram item claim process.mdj in the UML folder confirm this is a defined business process

C. Functional Requirements:

- Coordinate claim submission â†’ item status update â†’ officer notification
- Coordinate officer approval/rejection â†’ student notification â†’ handover trigger
- Provide claim status tracking across the full workflow
- Handle rollback if any step in the workflow fails

D. Non-Functional Requirements:

Attribute	Requirement
Security	Validates caller role at each step
Reliability	Compensating transactions on failure
Performance	Each step asynchronous where possible
Maintainability	Workflow steps loosely coupled via service interfaces

E. Suggested REST Resources:

```
POST /api/workflow/claim/submit      â€” Start claim workflow
GET  /api/workflow/claim/{id}/status â€” Get full workflow status
POST /api/workflow/claim/{id}/approve â€” Officer approval step
POST /api/workflow/claim/{id}/reject â€” Officer rejection step
POST /api/workflow/claim/{id}/handover â€” Trigger handover step
```

CS-2: Item Handover Orchestration Service

A. Description:
Coordinates the physical handover process: verifies student credentials, confirms the claim is approved, logs the handover, updates item status to Collected, and sends a completion notification. Evidence: HandoverController.logCollection(), HandoverController.verifyCollectionCredentials(), HandoverDataAccess, HandoverWindow.java, Template Method pattern (AbstractHandoverProcess, InPersonHandoverProcess).

- B. Why a Service Candidate:**
- The HandoverLog entity and dedicated HandoverWindow confirm this is a first-class business event
 - Requires coordination between Claim Service (status check), Item Service (status update), and Notification Service
 - The Template Method pattern already separates the handover algorithm from specific implementations

- C. Functional Requirements:**
- Verify that a student is authorized to collect a specific item
 - Log the physical handover with timestamp
 - Update item status to Collected
 - Send completion notification to student and officer

D. Non-Functional Requirements:

Attribute	Requirement
Security	Officer-initiated only; student identity verified
Reliability	Handover log persisted before status update
Auditability	Every handover permanently recorded in HANDOVER_LOG

E. Suggested REST Resources:

```
GET /api/handover/verify/{claimID}/{studentID} â€” Verify collection eligibility
POST /api/handover/log â€” Log handover completion
GET /api/handover/logs â€” Get all handover logs
GET /api/handover/logs/{claimID} â€” Get handover log for a claim
```

Section 4: Final Deliverable Tables

Table 1: Candidate Services Summary

#	Service Name	Type	Reusable	Independent	External Use Potential
US-1	Authentication Service	Utility	â€” High â€” all roles use it	â€” Fully stateless	â€” Mobile app, web portal login
US-2	Notification Service	Utility	â€” High â€” triggered by 3+ controllers	â€” Bridge pattern decouples it	â€” SMS/Email gateways
US-3	Password & Security Service	Utility	â€” Medium â€” shared by Auth + User Mgmt	â€” No business dependencies	â€” Internal only
BS-1	Found Item Service	Business	â€” High â€” Officers + Admin + Students	â€” Own DB table + lifecycle	â€” Officer mobile upload app
BS-2	Search & Matching Service	Business	â€” High â€” Students + Officers	â€” Read-only; decoupled	â€” Public search portal
BS-3	Claim Management Service	Business	â€” High â€” Students + Officers	â€” Own CLAIM table + strategies	â€” Student mobile claim app
BS-4	User Management Service	Business	â€” Medium â€” Admin only	â€” Multi-table role hierarchy	â€” Internal admin panel
BS-5	Reporting Service	Business	â€” Medium â€” Admin + Officers	â€” Analytical; separate from CRUD	â€” Dashboard / BI tool
BS-6	Archive Service	Business	â€” Low â€” Admin scheduled task	â€” Batch; no UI dependency	â€” Internal cron job
CS-1	Item Claim Workflow Service	Coordination	â€” High â€” orchestrates 4 services	â€” Depends on BS-1, BS-2, BS-3	â€” Full process API
CS-2	Item Handover Orchestration Service	Coordination	â€” Medium â€” final workflow step	â€” Depends on BS-1, BS-3	â€” Internal; Officer-initiated

Table 2: Functional & Non-Functional Requirements Summary

Service	Key Functional Requirements	Key Non-Functional Requirements
Authentication Service	Login, logout, token validation, user identity	Security (HTTPS, hashed passwords), 99.9% availability, <200ms response
Notification Service	Send (in-app/email/SMS), broadcast, history, mark-read	Async delivery, graceful degradation, retry on failure

Service	Key Functional Requirements	Key Non-Functional Requirements
Password & Security Service	Hash password, verify hash, generate token	Never log plaintext, algorithm configurable, <100ms
Found Item Service	CRUD items, photo upload, status transitions	Role-based auth, transactions for delete, <500ms listing
Search & Matching Service	Keyword search, multi-filter, strategy-based, scored results	<300ms response, read-only, indexed columns
Claim Management Service	Submit claim, list pending, approve/reject, track status	Atomic transactions, role enforcement, strategy-extensible
User Management Service	Create/delete user, update role, list/search users	Admin-only, transactional role changes, cascading deletes
Reporting Service	Stats, weekly/lost/claims reports, CSV export	<1 second generation, paginated for large datasets
Archive Service	Batch archive, configurable threshold, audit history	Non-blocking, atomic batch, admin-only
Item Claim Workflow	End-to-end claim orchestration, status tracking, rollback	Compensating transactions, async steps, role validation
Item Handover Service	Verify credentials, log handover, update status, notify	Officer-initiated, permanent audit log, reliable sequencing

Priority	Service	Reason
1 (Critical)	Authentication Service	Gateway to all system functions; no other service is accessible without it
2 (Critical)	Found Item Service	Core domain entity; all other services depend on items existing
3 (High)	Claim Management Service	Primary business process; directly serves Students and Officers
4 (High)	Search & Matching Service	Students cannot function without finding items first; feeds into claims
5 (High)	Notification Service	Required by Item upload and Claim approval flows; already architecturally designed (Bridge + Observer)
6 (Medium)	Item Claim Workflow Service	Reduces coupling between services; needed once BS-1, BS-2, BS-3 exist
7 (Medium)	User Management Service	Admin can manage users; needed for user onboarding
8 (Medium)	Item Handover Orchestration Service	Final step in workflow; needed once Claim Service is live
9 (Medium)	Reporting Service	Admin/Officer value; CSV export confirms real demand
10 (Low)	Password & Security Service	Critical for production security; should replace current plaintext storage before deployment
11 (Low)	Archive Service	Background maintenance; can be deferred until system has significant data

Best Services to Implement First (Deliverable #5)

Based on the repository analysis, the following **four services** are recommended for initial implementation:

Justification from code: `DBConnection.authenticateUser()` contains plain-text password comparison. Passwords stored as plaintext in `[User].PasswordHash`. The `hashPassword()` SHA-256 method is implemented but **commented out**. Exposing this as a secure REST service with proper JWT tokens and bcrypt hashing is the single highest-impact improvement.

This service is a **prerequisite for all other services** – no REST API can function without an authentication gateway.

Justification from code: `ItemController.registerNewItem()` is the most complete controller method in the system (validation, DB save, Bridge notification, Observer broadcast â€” all implemented and working). `ItemDataAccess` has full CRUD coverage. This service forms the **central data entity** that all other services reference via `ItemID`.

Exposing this as a REST service immediately enables mobile officers to log found items from the field.

Justification from code: ClaimController uses three concrete strategies (StrictClaimStrategy, FastClaimStrategy, ManualClaimStrategy). ClaimDataAccess has insertClaim, getPendingClaims, updateClaimStatus, getClaimById “all fully implemented. This directly serves the core business process: a student claiming their lost item.

Justification from code: MatchingController.performSmartSearch() is fully implemented with a 4-layer Decorator chain (Category â†’ Status â†’ Location â†’ Date). ItemDataAccess.searchByKeywordsAndCategory() is a working SQL query with parameterized filters. This is the **entry point for the student journey** â€” before a claim can be submitted, the student must find the item.

[illegible]

Assumptions Stated

1. The `HANDOVER_LOG` table exists in the database backup based on `HandoverLog.java` entity and `HandoverDataAccess.java` DAO, though its exact schema was inferred.
2. The `NotificationRecord` table or equivalent persistence layer is assumed to be needed for notification history; no explicit DB table was confirmed in SQL queries.
3. Item Matching score (`Item.matchScore()`) is stubbed â€” assumed to be a future feature for intelligent ranking.
4. The `ArchiveController.archiveExpiredItems()` is stubbed â€” Archive Service priority is low accordingly.
5. All claims belong to Students; the `CLaimedByUserID` column maps to `STUDENT.UserID` based on DAO evidence.

Report generated based on full repository analysis.
Evidence references: src/controller/, src/DAO/, src/entity/, src/Boundary/, src/bridge/notification/, src/behaviouralpatterns/, src/core/SessionManager.java, dataaccess/DBConnection.java.